

Tutorial 7 — Sequential Design, Registers, and Counters

Q1 — Binary Counters: Down, then Up-Down

Question (take home, not covered in class)

(a) 4-Bit Ripple (Asynchronous) Down Counter

Recall the 4-bit ripple up counter from lecture: $J = K = 1$ on every flip-flop (tied to logic-1, so each one always toggles on a clock edge), and each flip-flop is clocked directly by the Q output of the previous stage, negative-edge triggered (the bubble on each C input). No combinational logic is needed at all — the counting behavior comes entirely from how the flip-flops are clocked.

Design the corresponding 4-bit ripple (asynchronous) down counter the same way: $J = K = 1$ everywhere, each stage clocked from the previous stage's output. Instead of a direct Reset (which would initialize the counter to 0000), use a direct Set, so the counter initializes to 1111 and then counts down: 1111, 1110, 1101, ..., 0001, 0000, 1111, ...

What has to change from the up counter's clocking scheme for this to correctly count down?

(b) Up-Down Counter (synchronous)

This is a synchronous counter: every flip-flop shares the same clock, and the next state is found from combinational logic on the present state.

An n-bit up-down counter has a control input UD: UD = 0 counts up, UD = 1 counts down. For the 3-bit case (Q2 Q1 Q0), the state table is:

UD	Q2	Q1	Q0	Next Q2 Q1 Q0
0	0	0	0	001
0	0	0	1	010
0	0	1	0	011
0	0	1	1	100
0	1	0	0	101
0	1	0	1	110
0	1	1	0	111
0	1	1	1	000
1	0	0	0	111
1	0	0	1	000
1	0	1	0	001
1	0	1	1	010
1	1	0	0	011
1	1	0	1	100
1	1	1	0	101
1	1	1	1	110

Using Karnaugh maps, derive the flip-flop input equations two ways:

- 1. Using D flip-flops (Q2.D, Q1.D, Q0.D)
- 2. Using T flip-flops (T2, T1, T0)

Compare the total literal count between the two. Which is simpler, and why do you think that is, given what you already know about D vs. JK/T excitation tables? Draw the resulting circuit for whichever version (D-based or T-based) you found simpler.

Q2 — Counters with Unused States

Question (discuss in class)

A modulo-6 counter cycles through states 0, 1, 2, 3, 4, 5 (binary 000 through 101) and repeats. States 110 and 111 are never used in normal operation. Design this counter with JK flip-flops, two different ways. For each, work through the full process: draw the state diagram first, then build the state/excitation table from it, then derive the equations from K-maps.

(a) Unused states as don't-cares

Treat 110 and 111 as don't-care rows in your excitation table, and minimize normally.

(b) Unused states explicitly handled

Force both 110 and 111 to transition to state 000 on the next clock edge, and include this in your excitation table (no don't-cares for these rows).

(c)

For each of your two designs, suppose the counter somehow ends up in state 110 or 111 (e.g., due to power-on in an undefined state, or a glitch). Trace what actually happens over the next few clock cycles in each design. Does the don't-care version happen to recover? How many cycles does each design take to get back to the valid counting cycle, from each of the two unused states? Compare the gate cost of the two designs. What's the actual trade-off here?

Q3 — Coin-Operated Turnstile

Question (discuss in class)

A turnstile accepts one coin per cycle: a ₹5 coin (signal F, for “five”), a ₹10 coin (signal T, for “ten”), or no coin at all. (F and T are never both 1 in the same cycle.) The turnstile opens for exactly one cycle once ₹15 or more has accumulated, then resets to zero.

For example: ₹5 + ₹5 + ₹5 opens on the third coin. ₹10 + ₹10 opens on the second coin (with ₹5 of “change” implicitly forgiven — the turnstile just opens). ₹5 + ₹10 opens on the second coin.

(a)

Draw the state diagram. How many states do you need, and how many flip-flops does that require?

(b)

Assign binary codes to your states, build the excitation table (using JK flip-flops), and derive the flip-flop input equations and the output equation (Open) using K-maps.

(c)

Note that you likely have at least one unused state-code combination. How are you handling it here, and why?

Q4 — Bottle Inspection Line

Question (discuss in class)

A bottling line moves bottles past a fill-level sensor, Fill_Sensor, which reads 1 if the bottle currently at that position is underfilled.

Five positions downstream, a Reject_Arm must activate for exactly one cycle when an underfilled bottle (flagged five cycles earlier) reaches that position.

At the same downstream position, a separate sensor, Cap_Sensor, reads 1 if the bottle currently there has a missing cap. If Cap_Sensor reads 1 for three or more consecutive cycles, a Line_Stop alarm must turn on and stay on until Cap_Sensor reads 0 again.

Design the digital controller for this system. (Think about what kind of register handles the delay, and what kind of counter handles the consecutive-cycle detection — you don't need to derive full gate-level equations for this one, a clear structural design with the relevant equations for each block is enough.)

Q5 — A Combined Serial Adder/Subtractor

Question (optional — discuss in class if time permits, else take home)

Recall the serial adder from lecture: two shift registers hold A and B, a single full adder processes one bit pair per cycle, and a D flip-flop holds the carry between cycles.

Extend this into a combined add/subtract unit with a mode input M:

- $M = 0$: compute $A + B$
- $M = 1$: compute $A - B$

Use the standard two's-complement trick: feed the adder's Y input with $B \text{ XOR } M$ instead of raw B, and initialize the carry flip-flop to M (instead of always 0) at the start of each operation. (Convince yourself this is correct: when $M = 1$, this computes $A + (\text{one's complement of } B) + 1$, which is exactly $A + (\text{two's complement of } B) = A - B$.)

Using a JK flip-flop for the carry bit (state Q), with inputs X (bit of A), Y (bit of B), and M (mode, held constant for the whole operation):

(a)

Complete the state table (Q, X, M, and the effective Y after the XOR — or work directly with Q, X, Y, M as four input variables, whichever you find clearer) and derive J and K using K-maps.

(b)

Give the output equation for the sum/difference bit S.

(c)

Trace your design by hand for a small example: $A = 1001$ (9), $B = 0100$ (4), $M = 1$. Confirm you get 5.